SQL Data Warehouse Project

Project Requirements

Building the Data Warehouse (Data Engineering)

Objective: Develop a modern data warehouse using SQL Server to consolidate sales data, enabling analytical reporting and informed decision making.

Specifications

1. Data Sources: Import data from two source systems (ERP and CRM) provided as CSV files
2. Data Quality: Cleanse and resolve data quality issues prior to analysis.
3. Integration: Combine both sources into a single, user-friendly data model designed for analytical queries.
4. Scope: Focus on the latest dataset only, historization of data is not required.
5. Documentation: Provide clear documentation of the data model to support both business stakeholders and analytics teams

BI: Analytics & Reporting (Data Analysis)

Objective: Develop SQL-based analytics to deliver detailed insights into:

1. Customer Behavior
2. Product Performance
3. Sales Trends

These insights empower stakeholders with key business metrics, enabling strategic decision-making.

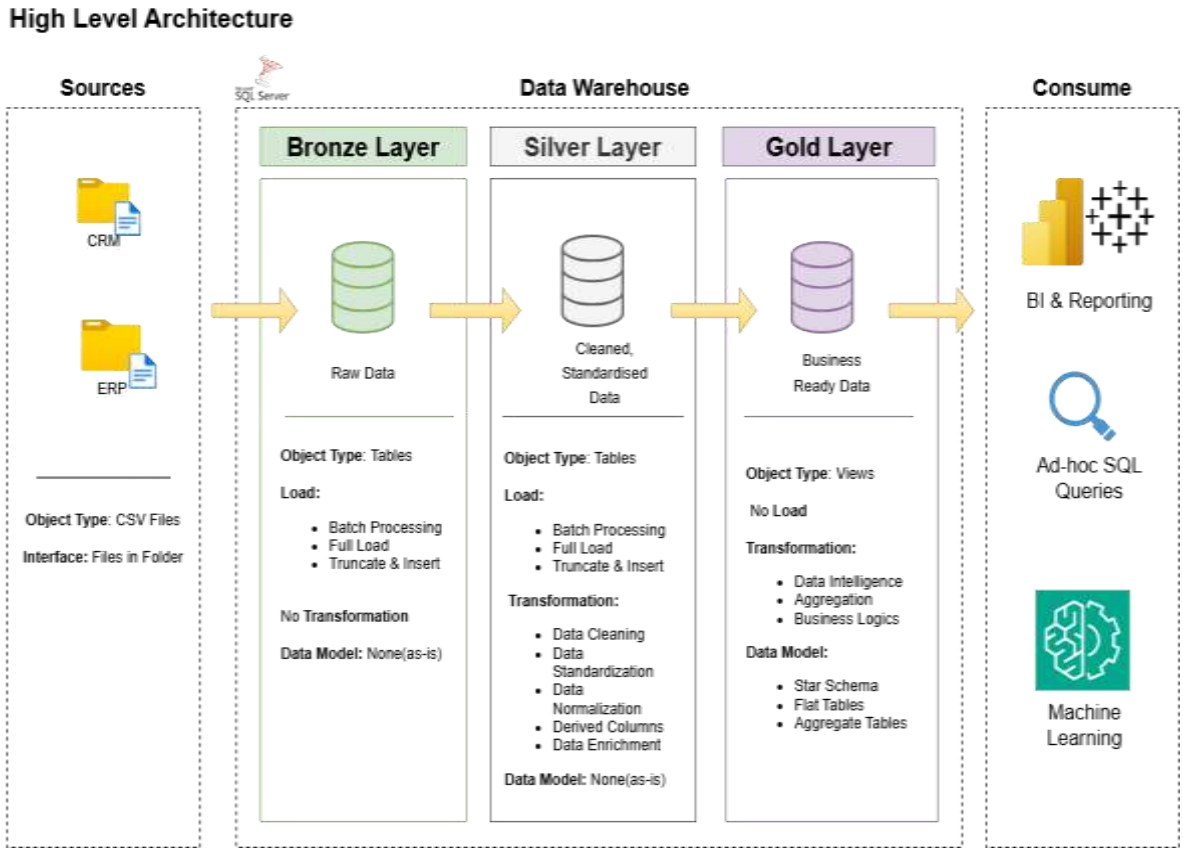
Data Management Approach Used: Medallion Architecture

Designing the Layers using Separation of Concerns(SOC)

	Bronze Layer	Silver Layer	Gold Layer
Definition	Row, unprocessed data as-is from sources	Clean & standardized Data	Business Ready data
Objective	Traceability & Debugging	(Intermediate Layer) Prepare Data for analysis	Provide data to be consumed for reporting and analytics
Object Type	Tables	Tables	Views
Load Method	Full Load (Truncate & Insert)	Full Load (Truncate & Insert)	None
Data Transformation	None (AS-IS)	-Data Cleaning -Data Standardization -Data Normalization -Derived Columns	-Data Integration -Data Aggregation -Business Logic & Rules

		-Data Enrichment	
Data Modelling	None (As-Is)	None (As-is)	-Start schema -Aggregate Objects -Flat Tables
Target Audience	Data Engineers	-Data Analyst -Data Engineers	-Data Analysis -Business Users

High Level Architecture



Naming Conventions:

- Using snake_case, with lowercase letters and underscores (_) to separate words.
- Language: Using English Language for all names
- Avoid Reserved Words: Not used SQL reserved words as an object names.

Table Naming Conventions:

Bronze Rules:

- All names must start with the source system name, and table names must match their original names without renaming.
- <sourcesystem>_<entity>
    a. <sourcesystem>: Name of the source system (eg:, crm, erp)
    b. <entity>: Exact table name from the source system.
    c. Example: crm_customer_info -> Customer information from the CRM system.

Silver Rules:

- All names must start with source system name, and table names must match their original names without renaming.
- <sourcesystem>_<entity>
    a. <sourcesystem>: Name of the source system (eg:, crm, erp)
    b. <entity>: Exact table name from the source system.
    c. Example: crm_customer_info -> Customer information from the CRM system.

Gold Rules:

- All names must use meaningful, business-aligned names for tables, starting with the category prefix.
- <category>_<entity>
    a. <category>: Describes the role of the table, such as dim (dimension) or fact (fact table).
    b. <entity>: Descriptive name of the table, aligned with the business domain (e.g., customers, products, sales)
    c. Examples:
        - dim_customers → Dimension table for customer data.
        - fact_sales → Fact table containing sales transactions.

Glossary of Category Patterns

Pattern	Meaning	Example(s)
dim_	Dimension table	dim_customer, dim_product
fact_	Fact Table	fact_sales
agg_	Aggregate Table	agg_customers, agg_sales_monthly

Column Naming Conventions

Surrogate Keys

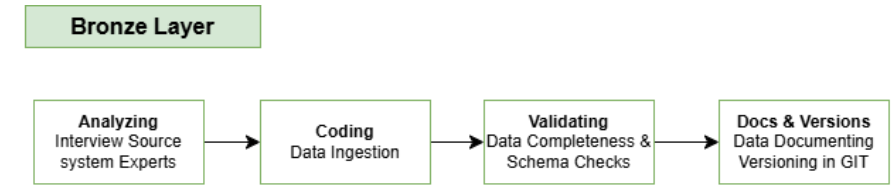
- All primary keys in dimension tables must use the suffix _key.
- <table_name>_key
    a. <table_name>: Refers to the name of the table or entity the key belongs to.
    b. _key: A suffix indicating that this column is a surrogate key.
    c. Example: customer_key → Surrogate key in the dim_customers table

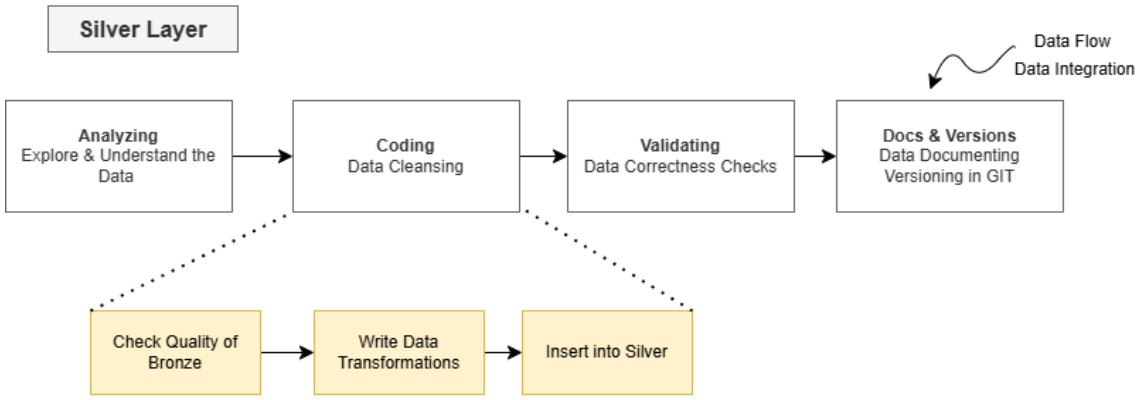
Technical Columns

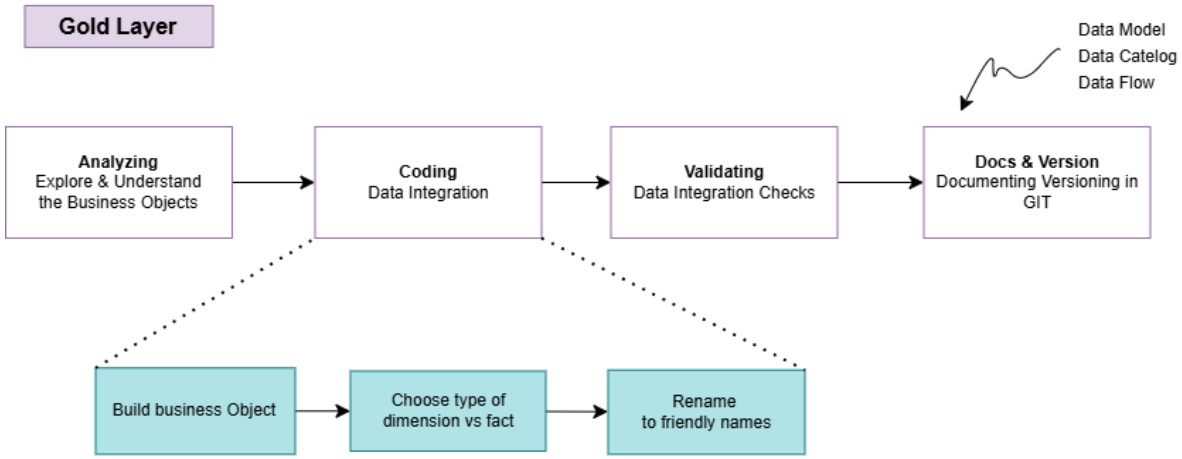
- All technical columns must start with the prefix dwh_, followed by the descriptive name indicating the columns purpose.
- dwh_<column_name>
    a. dwh: Preix exclusively for system-generated metadata.
    b. <column_name>: Descriptive name indicating the columns purpose.
    c. Example: dwh_load_date → System-generated column used to store the date when the record was loaded.

Stored Procedure

- All stored procedures used for loading data must follow the naming pattern: load_<layer>
    a. <layer>: Represents the layer being loaded, such as bronze, silver, or gold.
    b. Examples:
        - load_bronze → Stored procedure for loading data into the Bronze Layer.
        - load_silver → Stored procedure for loading data into the Silver Layer.
        - load_gold → Stored procedure for loading data into the Gold Layer.







Business Context & Ownership

Who owns the data?
What Business Process does it support?

System & Data documentation
Data Model & Data Catalog

Architecture & Technology Stack

How is data stored?
(SQL Server, Oracle, AWS, Azure, ...)

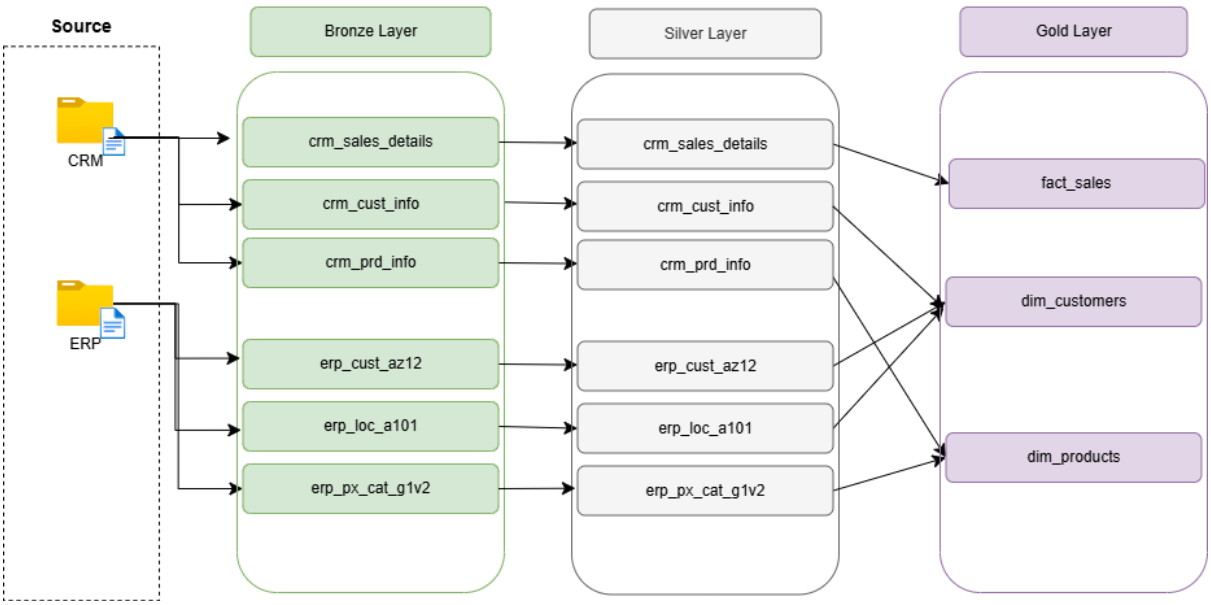
What are the Integration capabilities?
(API, Kafka, File Extract, Direct DB, ...)

Extract & Load

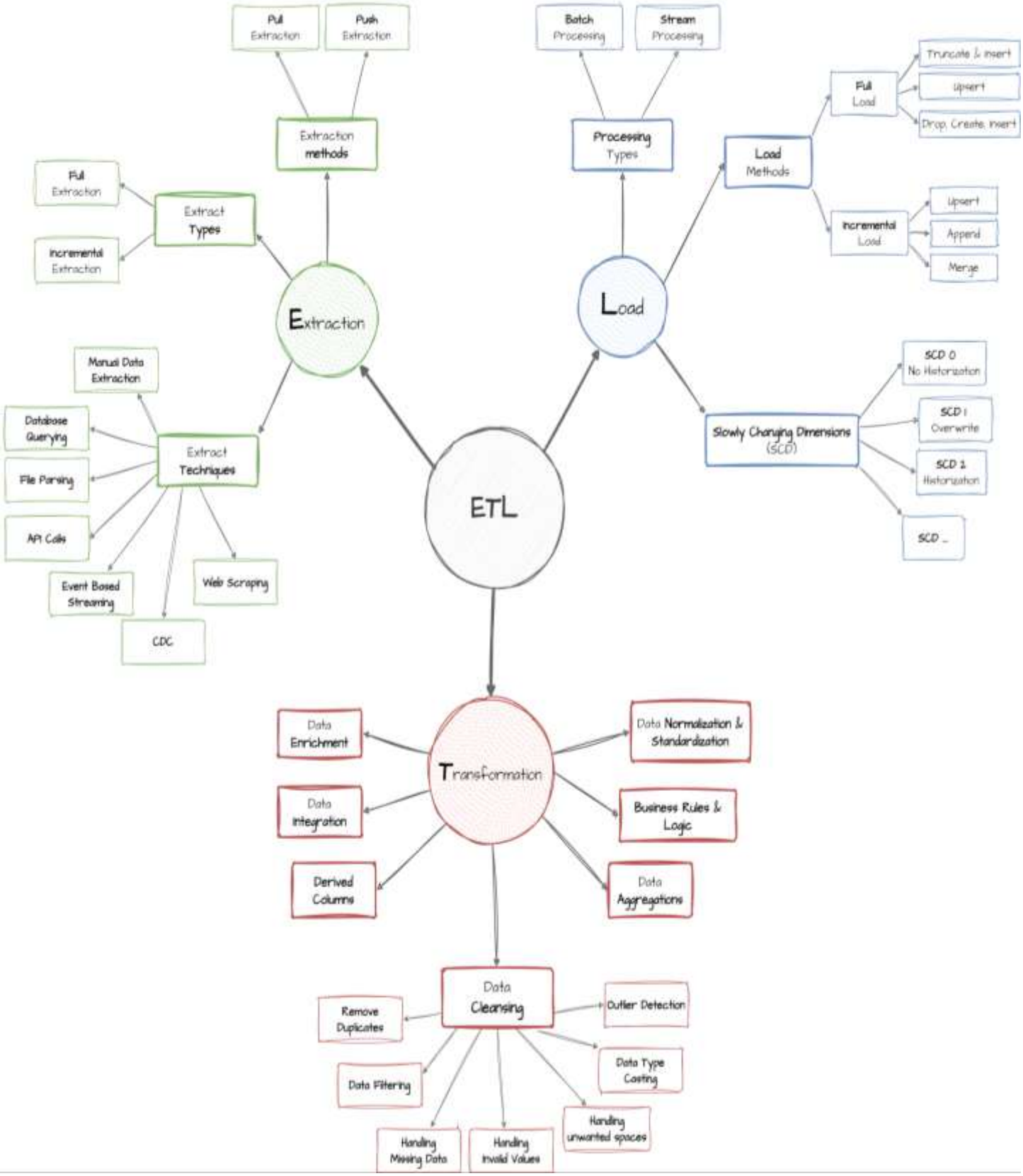
Incremental vs Full load?
Data Scope & Historical Needs
What is the expected size of the extracts?
Are there any data volume Limitations?
How to avoid impacting the source systems' performance?
authentications and authorization (tokens, SSH keys, VPN, IP whitelisting, ...)

I have used bulk Insert to load data in the database table.

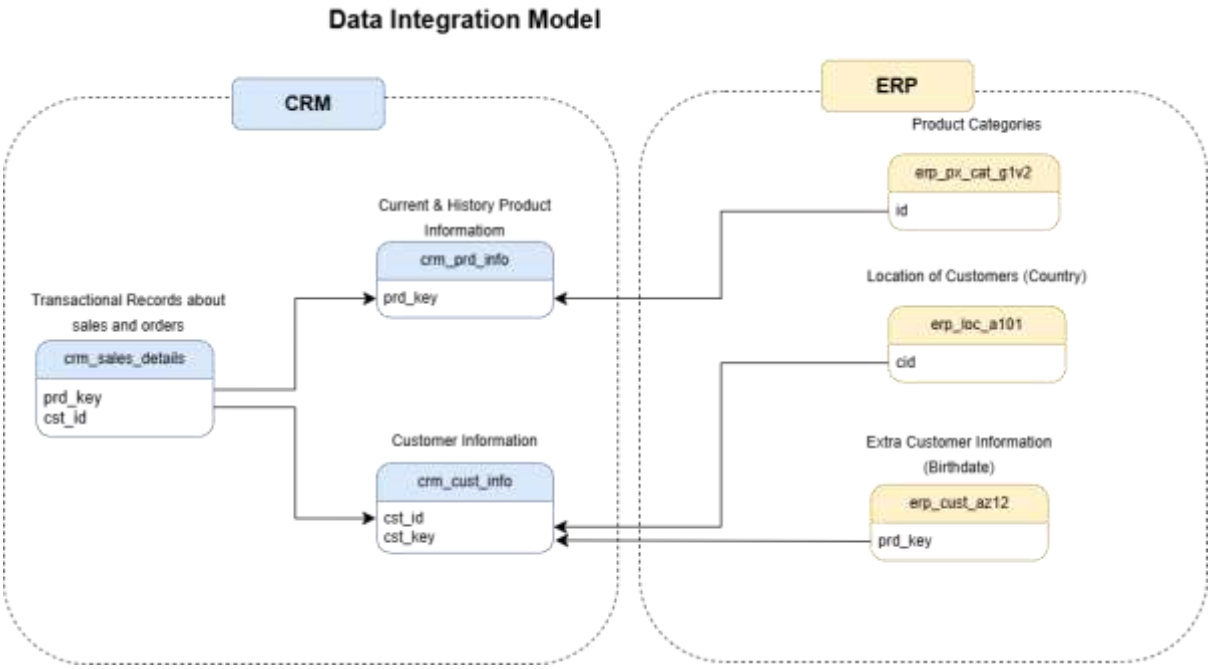
Data Flow Diagram (Data Lineage)



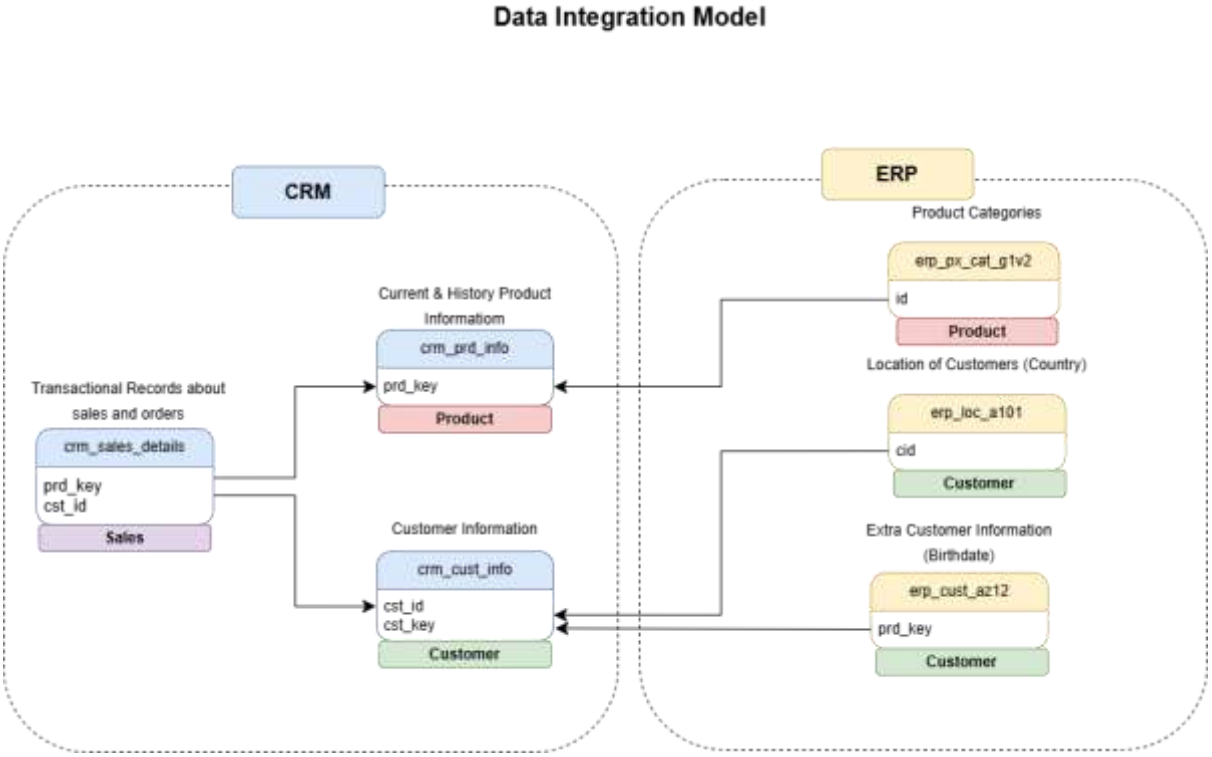
ETL


ETL
Extraction
Extraction methods
Pull Extraction
Push Extraction
Extract Types
Full Extraction
Incremental Extraction
Extract Techniques
Manual Data Extraction
Database Querying
File Parsing
API Calls
Event Based Streaming
CDC
Web Scraping
Load
Processing Types
Batch Processing
Stream Processing
Load Methods
Full Load
Truncate & Insert
Upsert
Drop, Create, Insert
Incremental Load
Upsert
Append
Merge
Slowly Changing Dimensions (SCD)
SCD 0 No Historization
SCD 1 Overwrite
SCD 2 Historization
SCD ...
Transformation
Data Enrichment
Data Integration
Derived Columns
Data Normalization & Standardization
Business Rules & Logic
Data Aggregations
Data Cleansing
Remove Duplicates
Data Filtering
Handling Missing Data
Handling Invalid Values
Outlier Detection
Data Type Casting
Handling unwanted spaces

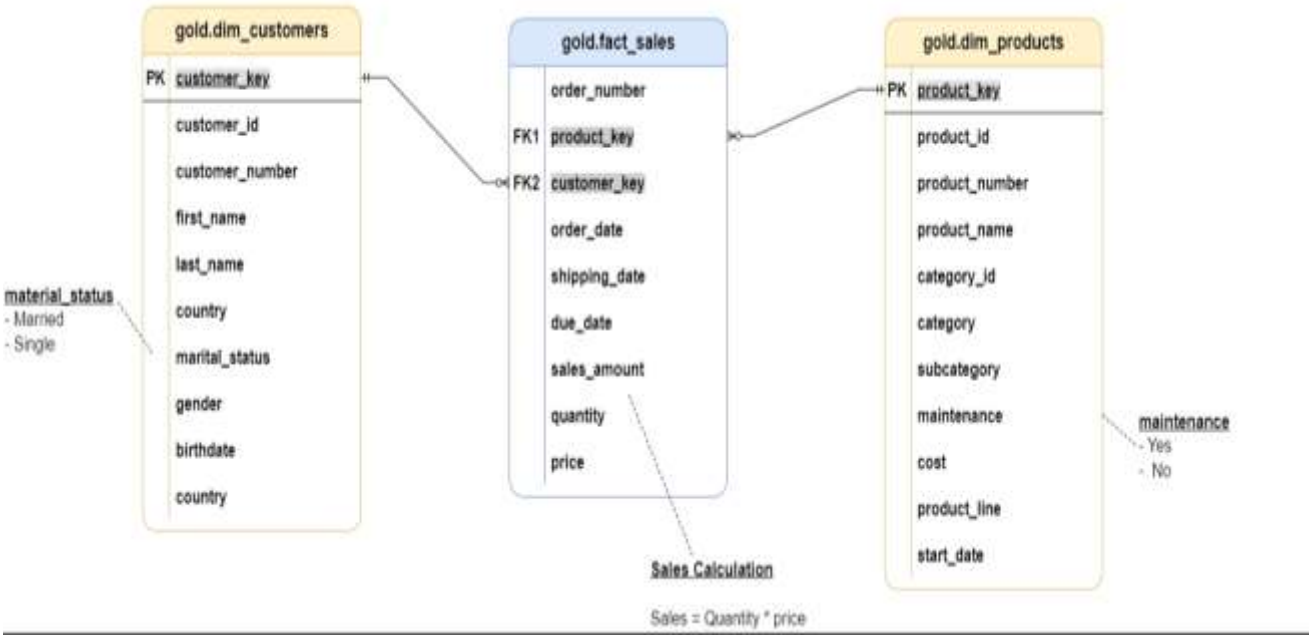
Integration Model



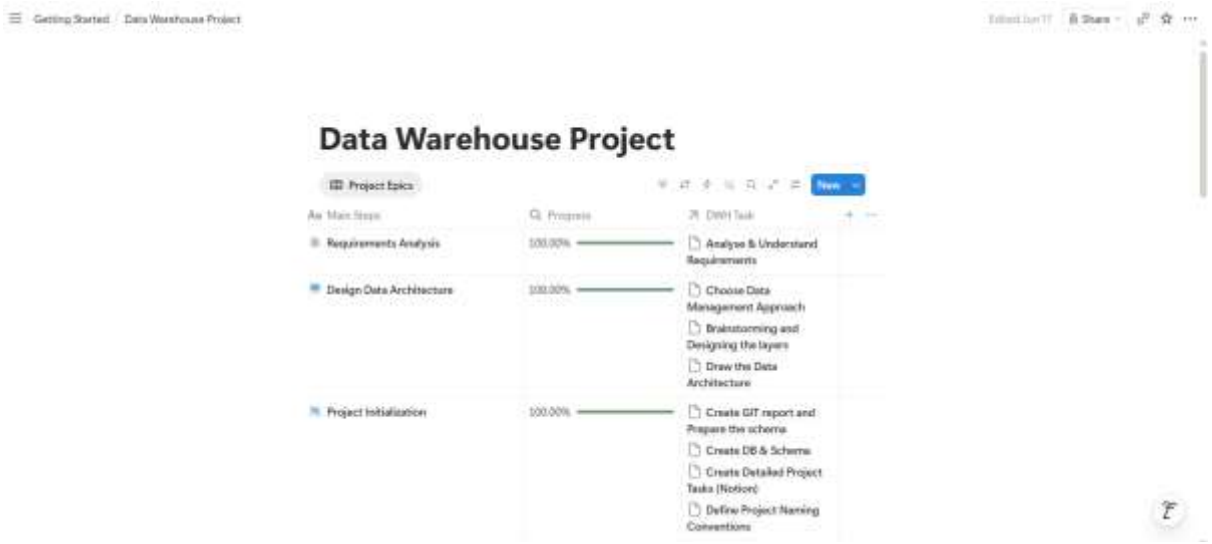
Data Model for gold layer



Sales Data Mart (Star Schema)


gold.dim_customers
PK customer_key
customer_id
customer_number
first_name
last_name
country
marital_status
gender
birthdate
country
gold.fact_sales
order_number
FK1 product_key
FK2 customer_key
order_date
shipping_date
due_date
sales_amount
quantity
price
gold.dim_products
PK product_key
product_id
product_number
product_name
category_id
category
subcategory
maintenance
cost
product_line
start_date
material_status
- Married
- Single
maintenance
- Yes
- No
Sales Calculation
Sales = Quantity * price

Notion


Data Warehouse Project
Project Epics
Requirements Analysis
Design Data Architecture
Project Initialization
Analyse & Understand Requirements
Choose Data Management Approach
Brainstorming and Designing the layers
Draw the Data Architecture
Create GIT repo and Prepare the schema
Create DB & Schema
Create Detailed Project Tasks (Notion)
Define Project Naming Conventions

Main Steps	Progress	DWH Task
Build Bronze Layer	100.00%	Analysing Source System Coding Data Integration Validating: Data Completeness & Schema Checks Document: Draw Data Flow (Draw.io) Commit Code in Git Repo
Build Silver Layer OPEN	100.00%	Analyzing: Explore & Understand Data Coding: Data Cleansing Validating: Data Correctness Checks Document: Draw data integration (draw.io) Commit Code in Git Repo Document: Extended Data Flow (Draw.io)
Build Gold Layer	100.00%	Analysing: Explore Business Objects Coding: Data Integration Validating: Data Integration Checks Document: Draw Data Model of Star Schema (Draw.io) Document: Create Data Catalog Document: Extended Data Flow (draw.io) Commit Code in GIT Repo

+ New page

Project Task

New

Requirements Analysis

DWH Epic	Tasks	
Requirements Analysis	Analyse & Understand Requirements	☑

New page

Design Data Architecture

DWH Epic	Tasks	
Design Data Architecture	Choose Data Management Approach	☑
Design Data Architecture	Brainstorming and Designing the layers	☑
Design Data Architecture	Draw the Data Architecture	☑

New page

▼ Project Initialization

DWH Epic	Tasks	
Project Initialization	Create DB & Schema	☑
Project Initialization	Create Detailed Project Tasks (Notion)	☑
Project Initialization	Define Project Naming Conventions	☑

+ New page

▼ Build Bronze Layer

DWH Epic	Tasks	
Build Bronze Layer	Analysing Source System	☑
Build Bronze Layer	Coding Data Integration	☑
Build Bronze Layer	Validating: Data Completene OPEN Schema Checks	☑
Build Bronze Layer	Document: Draw Data Flow (Draw.io)	☑
Build Bronze Layer	Commit Code in Git Repo	☑

+ New page

▼ Build Silver Layer

DWH Epic	Tasks	
Build Silver Layer	Analyzing: Explore & Understand Data	☑
Build Silver Layer	Document: Draw data integration (draw.io)	☑
Build Silver Layer	Coding: Data Cleansing	☑
Build Silver Layer	Validating: Data Correctness Checks	☑
Build Silver Layer	Commit Code in Git Repo	☑
Build Silver Layer	Document: Extended Data Flow (Draw.io)	☑

+ New page

▼ Build Gold Layer

DWH Epic	Tasks	
Build Gold Layer	Analysing: Explore Business Objects	☑
Build Gold Layer	Coding: Data Integration	☑
Build Gold Layer	Validating: Data Integration Checks	☑
Build Gold Layer	Document: Draw Data Model of Star Schema (Draw.io)	☑
Build Gold Layer	Document: Create Data Catalog	☑
Build Gold Layer	Document: Extended Data Flow (draw.io)	☑
Build Gold Layer	Commit Code in GIT Repo	☑

+ New page